

CSCE 641 COMPUTER GRAPHICS · MODULE 13 · LECTURE NOTES

Animation: Hierarchies, Skinning, and Interpolation

Making it move, and the mathematics of interpolating rotations.

MODULE OUTCOMES

Build a transform hierarchy and compute world transforms correctly.

Explain why Euler angles and matrices both interpolate badly.

Use quaternions and slerp for rotation interpolation.

Implement linear blend skinning and explain the candy-wrapper artifact.

Describe the animation pipeline from clip to final vertex.

1 Transform hierarchies

A skeleton is a tree in which each joint stores its transform relative to its parent. The reason is immediate: rotating a shoulder should carry the elbow, wrist, and fingers with it, and expressing each joint locally makes that automatic rather than something the animator has to maintain.

$$W_{\text{joint}} = W_{\text{parent}} \times L_{\text{joint}}$$

World transforms are computed by a depth-first traversal, parents before children. If joints are stored in topological order — which exporters generally guarantee — the traversal degenerates into a flat loop over an array, which is both simpler and far more cache-friendly than following pointers.

1.1 The bind pose

Mesh vertices are authored in a specific rest configuration: the **bind pose**, usually a T-pose or A-pose. The skeleton has a matching set of world transforms in that pose. To animate a vertex, you must first undo the bind pose and then apply the current pose:

$$M_{\text{skin}} = W_{\text{current}} \cdot W_{\text{bind}}^{-1}$$

THE INVERSE BIND MATRIX

W_{bind}^{-1} is precomputed once per joint and shipped with the mesh. It is the step most often omitted by people writing their first skinning implementation.

The symptom is unmistakable: the mesh explodes into scattered fragments the instant any joint is not at the origin, because every vertex is being transformed as though it were authored at the world origin.

If you see that, this is the cause. It is worth producing the broken result once deliberately.

2 Interpolating rotations

Animation is interpolation between keyframes. Position and scale interpolate linearly without difficulty. Rotation is the hard case, and the difficulty is intrinsic rather than an artifact of any particular representation.

2.1 Euler angles and gimbal lock

Three sequential rotations about three axes. Compact, intuitive to author, and degenerate. When the middle rotation reaches 90° , the first and third rotation axes become parallel: two of the three parameters now do the same thing, and one rotational degree of freedom cannot be expressed at all.

This is a topological fact, not a bug. There is no way to cover the rotation group smoothly with three parameters, so every three-parameter representation has a degeneracy somewhere. Euler angles remain a perfectly good *authoring* interface — artists think in pitch, yaw, and roll — and an unacceptable *interpolation* representation. Interpolating Euler angles also produces paths that depend on the chosen axis order and that are not shortest-arc.

2.2 Why matrices do not interpolate either

A less familiar problem: linearly interpolating two rotation matrices element-wise does not give a rotation matrix. The set of rotation matrices is a curved manifold embedded in the space of all 3×3 matrices, and it is not convex — the straight line between two points on it leaves it. Intermediate values therefore include shear and non-uniform scale, and at 180° apart the midpoint is degenerate. This is the same fact that will cause the candy-wrapper artifact in §3.

2.3 Quaternions

A unit quaternion is a point on the unit 3-sphere in four dimensions, and unit quaternions correspond to rotations. Interpolating *along the surface of the sphere* stays on the sphere, so every intermediate value is a valid rotation. **Slerp** traverses the shortest great-circle arc at constant angular velocity, which is exactly what rotational interpolation should mean.

```

quat slerp(quat a, quat b, float t) {
    float d = dot(a, b);

    // THE classic quaternion bug: q and -q are the same rotation, so a
    // negative dot product means the shortest path is toward -b. Without
    // this you can interpolate 358 degrees instead of 2.
    if (d < 0.0f) { b = -b; d = -d; }

    // Nearly parallel: sin(theta) -> 0 and the formula is unstable.
    // Plain lerp is indistinguishable here.
    if (d > 0.9995f) return normalize(a + t * (b - a));

    float theta = acosf(d);
    return (sinf((1-t)*theta)*a + sinf(t*theta)*b) / sinf(theta);
}

```

DOUBLE COVER

Every rotation corresponds to two quaternions, q and $-q$. The quaternion sphere covers the rotation group twice.

Practically this means a sign check before every interpolation. Omit it and a character's arm occasionally swings the wrong way round the entire circle between two nearly identical keyframes. It is dramatic, intermittent, and confusing until you know the cause — which is why it is worth meeting here rather than in production.

nlerp — linear interpolation followed by normalisation — is a common cheaper alternative. It stays on the sphere and is commutative, but moves at non-constant angular velocity. Between adjacent keyframes a few frames apart the difference is invisible, so many engines use nlerp for skeletal animation and reserve slerp for large interpolations such as camera moves.

3 Skinning

Rigid attachment of each vertex to one joint produces visible cracks and intersections at every joint. **Linear blend skinning** lets each vertex be influenced by several joints with weights that sum to one.

$$\mathbf{v}' = \sum_i \mathbf{w}_i \cdot \mathbf{M}_i \cdot \mathbf{v}, \quad \sum_i \mathbf{w}_i = 1$$

Four influences per vertex is the near-universal standard, because four weights fit in a vec4 and four indices in a uvec4, giving a tidy vertex layout. The weights must be normalised: if they do not sum to one the vertex is scaled toward or away from the origin, and the mesh visibly inflates or shrinks.

```
layout(location=5) in uvec4 aJoints;
layout(location=6) in vec4  aWeights;
layout(std430) buffer Skin { mat4 uSkin[]; }; // world * inverseBind

void main() {
    // Blend the MATRICES, then transform once. Transforming four times
    // and averaging the results gives the same answer for four times
    // the work.
    mat4 S = aWeights.x * uSkin[aJoints.x]
            + aWeights.y * uSkin[aJoints.y]
            + aWeights.z * uSkin[aJoints.z]
            + aWeights.w * uSkin[aJoints.w];

    vec3 pos = (S * vec4(aPos, 1.0)).xyz;
    vec3 nrm = mat3(S) * aNormal; // mat3: w = 0, Module 02
    gl_Position = uViewProj * vec4(pos, 1.0);
}
```

3.1 The candy-wrapper artifact

AVERAGING MATRICES IS NOT AVERAGING TRANSFORMATIONS

This is §2.2's problem arriving with consequences. The weighted average of two rotation matrices is not a rotation matrix.

For a 180° twist — a forearm, a wrist — the two joint matrices are nearly opposite, and their average is close to degenerate. The affected vertices collapse toward the bone axis, and the limb pinches to a point exactly like the twist in a sweet wrapper.

The artifact is proportional to the angle between the influencing joints, which is why it appears on wrists and shoulders and never on fingers.

Three responses, in increasing order of cost:

- 1 **Add joints.** Twist bones distribute a 180° rotation over several joints, so no single blend spans a large angle. Cheap, effective, and the standard production answer.
- 2 **Paint weights carefully.** Reduces but does not remove the problem, and is labour the rigger would rather spend elsewhere.

- 3 **Dual-quaternion skinning.** Represent each joint transform as a dual quaternion and interpolate *those* — interpolating transformations rather than their matrix representations. The collapse disappears entirely. The cost is a characteristic slight bulge at bent joints, and a somewhat more expensive vertex shader. Many engines offer both and let the rigger choose per mesh.

3.2 Normals and tangents deform too

A skinned vertex's normal must be transformed by the same skinning matrix, using only its upper 3×3 part so that translation does not reach it — Module 02's $w = 0$ rule. Strictly the inverse transpose is required, but since skinning matrices are usually rigid, the 3×3 part is correct and the full inverse transpose per vertex is not worth the cost.

The tangent frame deforms as well, which is the deeper reason Module 08 insisted on tangent-space normal maps: an object-space normal map would be wrong the moment the surface moved, while a tangent-space map remains correct because the frame it is relative to deforms with the surface.

4 The animation pipeline

Step	Where	What happens
Sample the clip	CPU	Find the keyframes bracketing the current time; lerp translation and scale, slerp or nlerp rotation.
Blend and layer	CPU	Blend between clips (walk to run) and layer additively (aim offset over a locomotion base). This is where a modern animation graph lives.
Inverse kinematics	CPU	Correct the result to meet constraints: feet on uneven ground, hands on a weapon, head tracking a target.
Hierarchy traversal	CPU	Compose local transforms into world transforms, parents first.
Build the palette	CPU	Multiply each world transform by that joint's inverse bind matrix; upload the array.
Skin	GPU	Blend matrices per vertex and transform position, normal, and tangent.

The first five steps cost roughly one hundred joints' worth of matrix arithmetic per character per frame — negligible. The last scales with vertex count and runs on the GPU. This is why animation systems can afford to be sophisticated on the CPU: the expensive part is elsewhere.

5 Where this leaves you

You have now built a renderer twice: once in software with nothing underneath it, and once on the GPU with a programmable pipeline. You can derive and compose transforms, rasterize with correct fill rules and perspective-correct interpolation, resolve visibility, shade with a physically based microfacet model under image-based lighting, cast and filter shadows, reason about aliasing as a sampling problem, texture with correct filtering and tangent-space normal maps, represent and simplify geometry, and animate a skinned

character.

That is the content of a graduate computer graphics course, and more importantly it is a working codebase. The remaining track courses each take one layer of it seriously: CSCE 647 replaces the rasterizer with real light transport; CSCE 645 goes properly into the geometry; CSCE 649 replaces keyframes with physics; CSCE 650 addresses the perceptual constraints of stereo and latency; CSCE 735 explains the machine the whole thing runs on.

THE CAPSTONE

Build it on this codebase. A renderer you wrote from the triangle up, extended into a research-scale project, with a written report and a recorded demonstration, is a more legible credential in this field than a transcript.

That is not a consolation for the lack of accreditation. In graphics it is genuinely how people are evaluated.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lectures 19–20 — Animation	Keyframing, rigging, and skinning at the right level, with the rotation-interpolation problem shown clearly.
3Blue1Brown & Ben Eater — 'Visualizing quaternions' (interactive)	The best available intuition for why quaternions represent rotations. Interactive and free.
Kavan et al. — 'Skinning with Dual Quaternions' (free)	The paper that fixed candy-wrapping, with the derivation and the honest account of the bulging trade-off.
glTF 2.0 specification — skins and animations	The standard runtime format. Reading the skinning section is the fastest way to see exactly what data a real pipeline ships.
Jason Gregory — Game Engine Architecture, animation chapter (sample chapters free)	How animation systems are actually structured in production engines.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a transform hierarchy with world-transform computation. Verify that rotating a parent joint carries all descendants correctly.

- 2 Load a skinned glTF model. Implement linear blend skinning in the vertex shader and play back an animation clip.
- 3 Deliberately omit the inverse bind matrix and capture the result. Then fix it. Write one sentence describing the artifact precisely enough to recognise it instantly next time.
- 4 Implement slerp. Then remove the $d < 0$ sign check and find two keyframes where the rotation takes the long path. Record both.
- 5 Build a wrist twisting through 180° with two joint influences and capture the candy-wrapper collapse. Then add a twist bone and capture again.
- 6 Compare slerp and nlerp on the same animation. Measure both and determine experimentally at what angular separation the difference becomes visible.
- 7 **Course complete.** Assemble your portfolio: both projects, every module's output image, and a README describing what you built and what was hard.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How is a joint's world transform computed, and why must joints be processed in topological order?
- 2 What does the inverse bind matrix do, and what exactly goes wrong without it?
- 3 Explain gimbal lock, and why it is a topological fact rather than an implementation flaw.
- 4 Why does linear interpolation of two rotation matrices not produce a rotation?
- 5 What does slerp do, and what is the double-cover problem? What does the bug look like?
- 6 Explain the candy-wrapper artifact, its cause, and three mitigations with their trade-offs.
- 7 Why does skinning make tangent-space normal maps necessary rather than merely convenient?